

Contrôler les animations avec JavaScript

Dans cette leçon, nous allons voir comment contrôler les animations avec JavaScript. Nous allons voir comment démarrer, arrêter, mettre en pause et reprendre une animation. Nous allons également voir comment gérer les événements liés aux animations. Finalement, nous allons voir comment décaler l'exécution d'une fonction.

Temporisation de fonctions

setTimeout

Pour exécuter une fonction après un certain délai, vous pouvez utiliser la fonction `setTimeout`. Cette fonction appartient à l'objet `window` et prend deux arguments : la fonction à exécuter et le délai en millisecondes. Cette fonction n'est exécutée qu'une seule fois.

```
let elementHTML = document.body;
elementHTML.classList.add("invisible");

function afficher() {
  elementHTML.classList.remove("invisible");
}

setTimeout(afficher, 1000); // Affiche "Hello !" après 1 seconde
```

setInterval

Pour exécuter une fonction à intervalles réguliers, vous pouvez utiliser la fonction `setInterval`. Cette fonction appartient également à l'objet `window` et prend deux arguments : la fonction à exécuter et l'intervalle en millisecondes.

```
let elementHTML = document.body;

function clignoter() {
  elementHTML.classList.toggle("invisible");
}

let intervalle = setInterval(clignoter, 1000); // Fait clignoter l'élément toutes les secondes
// Il est important de l'enregistrer dans une variable pour pouvoir l'arrêter plus tard
```

clearInterval

Pour arrêter l'exécution d'une fonction à intervalles réguliers, vous pouvez utiliser la fonction `clearInterval`. Cette fonction prend en argument l'identifiant de l'intervalle retourné par `setInterval`.

```
let elementHTML = document.body;
let compteur = 0;
function clignoter() {
  compteur++;
  if (compteur === 5) {
    clearInterval(intervalle); // Arrête le clignotement après 5 fois
  }
  elementHTML.classList.toggle("invisible");
}

let intervalle = setInterval(clignoter, 1000); // Exécute la fonction toutes les
secondes
```

Contrôler les transitions CSS

Pour contrôler les transitions CSS avec JavaScript, vous pouvez utiliser les propriétés `transition` et `transitionend`. L'objectif est d'attendre la fin d'une transition pour exécuter une fonction. Sans ce délai, la fonction serait exécutée immédiatement, ce qui pourrait entraîner des problèmes d'affichage.

Dans l'exemple suivant, nous allons ajouter une classe à un élément pour le rendre invisible, puis attendre la fin de la transition pour le supprimer du DOM.

La transition démarre au moment où vous ajoutez la classe à l'élément.

```
.invisible {
  opacity: 0;
  transition: opacity 1s;
}
```

```
let elementHTML = document.querySelector("section");

elementHTML.addEventListener("transitionend", function () {
  elementHTML.remove();
});

elementHTML.classList.add("invisible");
```

Contrôler les animations CSS

Il est possible de contrôler les animation de la même manière que les transitions. Pour cela, vous pouvez utiliser les propriétés `animation` et `animationend`. L'avantage des animations par rapport aux transitions est que vous pouvez définir des étapes intermédiaires pour l'animation.

L'animation démarre au moment où vous ajoutez la classe à l'élément.

```
@keyframes disparition {
  from {
    opacity: 1;
  }
  to {
    opacity: 0;
  }
}

.disparition {
  animation: disparition 1s;
}
```

```
let elementHTML = document.querySelector("section");
elementHTML.addEventListener("animationend", disparaitre);

function disparaitre() {
  // elementHTML.removeEventListener("animationend", disparaitre); // Optionnel
  elementHTML.classList.remove("disparition");
}

elementHTML.classList.add("disparition");
```

Redémarrer une animation CSS

Pour redémarrer une animation CSS, vous devez enlever la classe qui la déclenche, puis la remettre. Cela permet de forcer le navigateur à redémarrer l'animation. Cependant, il faut attendre que l'animation soit terminée pour la redémarrer car en enlevant la classe, l'animation est stoppée, d'où l'intérêt de l'écoute de l'événement `animationend`.